

Overview

Advanced NLP today is dominated by transformer-based language models, large-scale pretraining, alignment techniques, and application-layer orchestration. Beyond basic prompting, production NLP/LLM systems require robust conditioning, retrieval, tool integration, and safety/evaluation loops.

Core progression:

- language modeling foundations,
- pretraining objectives,
- adaptation (fine-tuning/instruction tuning/alignment),
- system-level evaluation and guardrails.

Advanced NLP Tasks & Methods

Sequence-to-Sequence Modeling

Seq2seq models map input sequence to output sequence:

- machine translation,
- summarization,
- paraphrasing,
- structured report generation.

Transformer encoder-decoder models (for example T5-like) are common in this setting.

Pretraining Objectives

Masked Language Modeling (MLM)

Predict masked tokens from bidirectional context.

Strength:

- strong contextual representations.

Typical in encoder-style models.

Causal Language Modeling (CLM)

Predict next token autoregressively.

Strength:

- natural text generation and completion.

Typical in decoder-style models.

Adaptation Methods

Fine-Tuning

Supervised adaptation on task-specific labeled data.

Instruction Tuning

Train on instruction-response pairs to improve instruction following and general helpfulness.

RLHF (High-Level)

Reinforcement learning from human feedback typically uses preference data to align model behavior toward helpful, safe, and policy-compliant outputs.

Modern LLM Application Patterns

Structured Outputs

Constrain model outputs to JSON schemas or typed formats for downstream reliability.

Tools and Function Calling

LLM delegates operations to external tools (search, database, calculators, APIs) and composes final response from verified outputs.

Multi-Step Workflows

Patterns include:

1. retrieve -> reason -> synthesize,
2. planner -> executor -> verifier,
3. prompt-eval iteration loops.

These patterns connect directly with agentic architectures from Lesson 7.

Evaluation & Safety in NLP/LLMs

Evaluation Dimensions

1. task quality (accuracy, ROUGE/BLEU/BERTScore as applicable)
2. groundedness/citation correctness
3. robustness (adversarial prompts, OOD behavior)
4. latency/cost per request
5. human preference and usability

Safety Risks

- toxicity and harmful outputs,
- hallucinations and factual errors,
- jailbreak susceptibility,
- data leakage/privacy violations.

Practical Safety Controls

- prompt and output filtering,
- retrieval grounding,
- model routing and fallback,
- human review for high-risk workflows,
- red-teaming and adversarial test sets.

Business Case Studies & Exceptions

Case Study 1: Enterprise Summarization

Scenario:

- Legal operations team summarizes long contracts and policy updates.

Architecture:

- chunking + retrieval + seq2seq summarization,
- structured output template,
- citation links to source passages.

Trade-offs:

- concise summaries may omit nuance; human review required for legal-critical sections.

Case Study 2: Developer Code Assistant

Scenario:

- Internal coding assistant supports bug triage and documentation.

Benefits:

- reduced context-switching and faster onboarding.

Risks:

- insecure code suggestions,
- hallucinated APIs,
- license/compliance concerns.

Mitigation:

- static analysis gates,
- repository-aware retrieval,
- policy checks before merge.

Exceptions

- If domain requires strict determinism (for example numeric compliance calculations), symbolic/rule-based systems may be better.
- For low-data specialized tasks, small supervised models can outperform general LLM pipelines on cost and controllability.

Interview Questions & Answers

1. **Explain masked language modeling.**
MLM masks tokens and trains the model to predict them using bidirectional context.
2. **Explain causal language modeling.**
CLM predicts the next token given previous tokens in autoregressive generation.
3. **Fine-tuning vs instruction tuning?**
Fine-tuning targets specific tasks; instruction tuning improves broad instruction-following behavior.
4. **What is RLHF and why use it?**
Alignment method using human preference signals to steer outputs toward helpful/safe behavior.

5. **What makes seq2seq suitable for summarization?**
It naturally maps long input text to shorter target output with conditional generation.
6. **Why do LLM apps need structured outputs?**
To improve reliability and integration with downstream services.
7. **How do you evaluate LLM applications?**
Combine automatic metrics, human evaluation, groundedness checks, and operational metrics.
8. **What is hallucination in LLM systems?**
Confident generation of unsupported or incorrect content.
9. **How can retrieval reduce hallucinations?**
It injects relevant evidence that constrains generation.
10. **What is a jailbreak?**
Prompt strategy attempting to bypass safety constraints.
11. **How do you test robustness?**
Adversarial prompts, perturbation tests, and scenario-based stress suites.
12. **When would a smaller model be preferred?**
When latency/cost/privacy constraints dominate and task complexity is moderate.
13. **How do you handle sensitive data in LLM pipelines?**
PII redaction, scoped access, logging controls, and policy-compliant retention.
14. **Why is human evaluation still important?**
Automated metrics often miss usefulness, nuance, and domain correctness.
15. **How do you productionize a summarization system?**
Add chunking/retrieval, structured templates, evaluation harnesses, and monitoring + fallback.

References

- Stanford CS224N: <https://web.stanford.edu/class/cs224n/>
- CS224N bulletin entry: <https://bulletin.stanford.edu/courses/1209041>
- Hugging Face summarization task docs: <https://huggingface.co/docs/transformers/v5.7.0/tasks/summarization>
- T5 model card: <https://huggingface.co/google-t5/t5-small>
- CS231n and CS285 context for transformer/RL crossover: <https://cs231n.stanford.edu/> and <https://www2.eecs.berkeley.edu/Courses/CS285/>